# Big Data Analytics

Tuần 3 — Phần 2: Lưu trữ phân tán & Lakehouse

*HDFS, Google File System, định dạng file cột, Data Lake/Warehouse/Lakehouse*

# Nhắc lại: chúng ta đang ở đâu trong roadmap?




*Ở Tuần 2 (Phần 1), chúng ta đã học **data mining là gì**, mô hình **5V**, và lý do vì sao **Secondary Storage** (đĩa cứng) là một giới hạn vật lý cần thiết kế thuật toán để né tránh.*

- Câu hỏi tiếp theo: nếu dữ liệu quá lớn để lưu trên **một** máy, thì lưu trữ nó **phân tán trên nhiều máy** như thế nào?
- Và một khi đã có hàng petabyte dữ liệu thô, làm sao tổ chức chúng để vừa **rẻ**, vừa **đáng tin cậy**, vừa **dễ truy vấn**?
- → Đây chính là chủ đề của Phần 2: **Lưu trữ phân tán & Lakehouse**

# Agenda Phần 2

- Lý thuyết:
  - HDFS — kiến trúc, nguyên lý thiết kế, block & replication
  - Google File System (GFS) — nền tảng học thuật của HDFS
  - Chịu lỗi & đặt bản sao (fault tolerance, replica placement)
  - Định dạng file: row-based vs. columnar (Avro, Parquet, ORC)
  - Lưu trữ đối tượng trên Cloud (S3, GCS, Azure Blob) vs. HDFS
  - Data Warehouse, Data Lake, Lakehouse — so sánh & kiến trúc
  - Delta Lake, Apache Iceberg, Apache Hudi
- Thực hành:
  - Bài tập tính toán dung lượng lưu trữ với replication
  - Đọc code HDFS Java API, mô phỏng rack-aware placement bằng Python
  - So sánh CSV vs. Parquet bằng Python (kích thước, tốc độ đọc)
  - Khám phá transaction log của Delta Lake, so sánh Delta/Iceberg/Hudi

# Mục tiêu học tập

- Giải thích được **vì sao** và **như thế nào** HDFS/GFS phân tán dữ liệu ra nhiều máy mà vẫn đảm bảo độ tin cậy
- Phân biệt được **row-based** và **columnar storage**, biết khi nào dùng định dạng nào
- Phân biệt rạch ròi **Data Warehouse**, **Data Lake**, **Lakehouse** — và giải thích được vì sao Lakehouse ra đời
- Biết được vai trò của **transaction log** trong Delta Lake/Iceberg/Hudi để đưa ACID vào Data Lake

# Vị trí Phần 2 trong roadmap 7 phần

- Phần 2 là **nền tảng lưu trữ** cho mọi phần phía sau
- P3 (MapReduce/Spark) đọc/ghi trực tiếp trên HDFS hoặc object storage đã học ở đây
- P5 (NoSQL) là một dạng lưu trữ phân tán khác, chuyên biệt hơn cho truy vấn theo khóa
- P6 (Spark MLlib) thường đọc dữ liệu huấn luyện từ các bảng Lakehouse (Delta/Iceberg)



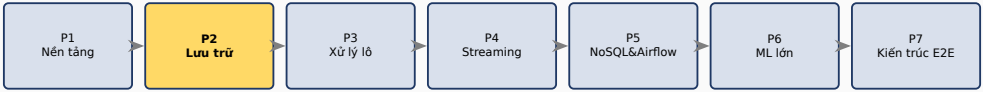
# Nguồn tài liệu tham khảo chính cho Phần 2

- **Apache Hadoop — HDFS Architecture Guide** (docs chính thức): hadoop.apache.org
- **"The Google File System"** (Ghemawat, Gobioff, Leung — SOSP 2003), PDF chính thức từ Google Research
- **"Lakehouse: A New Generation of Open Source Systems and Applications"** (Armbrust et al. — CIDR 2021), PDF mở từ CIDR
- **Apache Parquet, Delta Lake, Apache Iceberg** — tài liệu chính thức của từng dự án
- Code mẫu: *Hadoop: The Definitive Guide* (repo GitHub của tác giả), Delta Lake examples, Iceberg + Spark quickstart — đã tải về trong `TaiLieuThamKhao/P2_LuuTruPhanTan/`

# PHẦN 2

# HDFS — HADOOP DISTRIBUTED FILE SYSTEM

*Hệ thống file phân tán làm nền tảng lưu trữ cho toàn bộ hệ sinh thái Hadoop/Spark*



# Vì sao cần một hệ thống file phân tán?

- Nhắc lại Tuần 2: một đĩa cứng đơn có **giới hạn dung lượng** và **giới hạn băng thông đọc/ghi** — không thể chứa/đọc hàng petabyte dữ liệu trong thời gian hợp lý
- Giải pháp **scale-out**: chia file lớn thành nhiều mảnh (block), rải đều trên **hàng trăm–hàng nghìn máy**, đọc/ghi **song song**
- Thách thức mới nảy sinh: với hàng nghìn máy, **lỗi phần cứng không còn là ngoại lệ mà là chuyện thường ngày**
- HDFS (Hadoop Distributed File System) được thiết kế riêng để giải quyết đồng thời cả hai bài toán: mở rộng quy mô **và** chịu lỗi

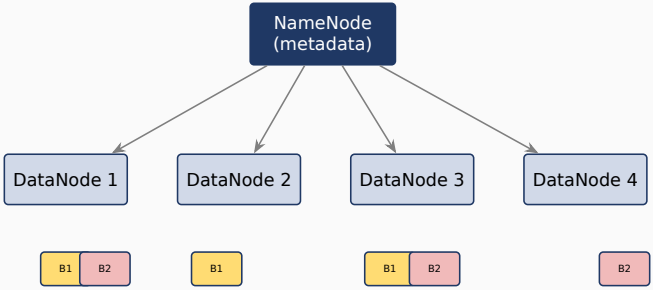
# HDFS — các giả định thiết kế nền tảng (1/2)

- Hardware failure là chuyện bình thường
  - Nguyên văn tài liệu chính thức: "hardware failure is the norm rather than the exception"
  - Với cụm hàng nghìn máy, luôn có **một thành phần nào đó** đang hỏng — phát hiện lỗi & tự phục hồi là ưu tiên kiến trúc hàng đầu
- Streaming data access
  - Ưu tiên **thông lượng cao** (high throughput) hơn **độ trễ thấp** (low latency)
  - Đánh đổi một số ngữ nghĩa POSIX để đạt tốc độ đọc/ghi dữ liệu lớn tối ưu

# HDFS — các giả định thiết kế nền tảng (2/2)

- Large data sets
  - File có kích thước điển hình từ **gigabyte đến terabyte**
  - Một cụm HDFS hỗ trợ **hàng chục triệu file**, băng thông tổng hợp cao trên hàng trăm node
- Simple coherency model
  - Mô hình **write-once-read-many**: file sau khi ghi và đóng lại thì không đổi (trừ append) — đơn giản hóa bài toán đồng bộ
- Di chuyển tính toán rẻ hơn di chuyển dữ liệu
  - "Moving computation is much more efficient" khi thực thi **ngay tại nơi có dữ liệu** — nền tảng của mô hình MapReduce (Tuần 4)

# HDFS — kiến trúc tổng quan: NameNode & DataNode




Mỗi block (128MB) được nhân bản 3 lần trên các DataNode khác nhau

*Kiến trúc master–slave: một NameNode quản lý metadata, nhiều DataNode lưu trữ dữ liệu thực tế*

# HDFS — vai trò của NameNode

- **NameNode** (master) quản lý **namespace** của hệ thống file: cây thư mục, tên file, và **ánh xạ file → danh sách block**
- **Không** lưu trữ dữ liệu thực tế — chỉ lưu metadata (nhẹ, có thể giữ toàn bộ trong RAM để truy vấn nhanh)
- Là **điểm truy vấn duy nhất** mà client hỏi trước khi đọc/ghi: "file X nằm ở block nào, block đó nằm trên DataNode nào?"
- Vì chỉ có **một** NameNode chính, đây từng là **single point of failure** — các phiên bản HDFS hiện đại có NameNode dự phòng (High Availability) để khắc phục

# HDFS — vai trò của DataNode

- **DataNode** (worker) chạy trên mỗi máy trong cụm, lưu trữ các **block dữ liệu thực tế** trên đĩa cục bộ
- Định kỳ gửi **heartbeat** và **báo cáo danh sách block** (block report) về cho NameNode — đây là cách NameNode biết cụm còn "khỏe" hay không
- Client đọc/ghi dữ liệu **trực tiếp** với DataNode (không qua NameNode) — NameNode chỉ đóng vai trò "chỉ đường"
- Thiết kế này giúp NameNode **không trở thành nút thắt cổ chai** (bottleneck) khi hàng nghìn client đọc/ghi dữ liệu đồng thời

# HDFS — Block size & Replication factor

Một file lớn được chia thành các **block** có kích thước cố định:

$$\text{block size mặc định} = 128\text{MB}$$

So sánh với block đĩa cứng truyền thống (Tuần 2 đã học: 4KB–64KB) — lớn hơn **hàng nghìn lần**. Vì sao? Block lớn giúp *giảm số lần truy vấn NameNode* và giữ được ưu điểm đọc *tuần tự* (sequential) đã học ở bài Secondary Storage.

Mỗi block được nhân bản (replication) mặc định **3 lần** trên các DataNode khác nhau:

$$\text{replication factor} = 3$$

Đây là cơ chế chịu lỗi chính của HDFS — mất 1–2 bản sao vẫn còn dữ liệu.

# HDFS — ví dụ số: chi phí lưu trữ thực tế của replication

- Giả sử có **10 TB** dữ liệu thô cần lưu trên HDFS, với replication factor mặc định $= 3$
- Dung lượng **vật lý thực tế** cần dùng: $10\text{TB} \times 3 = 30\text{TB}$
- $\Rightarrow$ **Storage overhead** $= 200\%$ — đây là cái giá phải trả để đổi lấy độ tin cậy cao
- So sánh: nhiều hệ thống lưu trữ hiện đại (ví dụ Erasure Coding trong HDFS 3.x) giảm overhead xuống còn khoảng $\sim 50\%$ bằng mã sửa lỗi thay vì nhân bản toàn bộ — đánh đổi thêm chi phí tính toán khi phục hồi

# HDFS — quy trình ghi file (Write Pipeline)

- 1. Client hỏi NameNode: "tôi muốn tạo file mới, cho tôi danh sách DataNode để ghi block đầu tiên"
- 2. NameNode chọn 3 DataNode (theo chính sách rack-aware — xem slide sau) và trả về một **pipeline**
- 3. Client ghi dữ liệu tới DataNode đầu tiên; DataNode 1 vừa ghi vừa **chuyển tiếp** (forward) dữ liệu sang DataNode 2, rồi DataNode 2 chuyển tiếp sang DataNode 3
- 4. Khi cả 3 DataNode xác nhận ghi thành công, **acknowledgement** được gửi ngược lại client — block được xem là đã ghi an toàn
- $\Rightarrow$ Dữ liệu chỉ truyền **một lần** từ client, nhân bản diễn ra **giữa các DataNode** — tiết kiệm băng thông phía client

# HDFS — quy trình đọc file (Read Pipeline)

- 1. Client hỏi NameNode vị trí các block của file cần đọc
- 2. NameNode trả về danh sách DataNode chứa từng block, **sắp xếp theo khoảng cách mạng** tới client (gần nhất trước)
- 3. Client đọc trực tiếp từ DataNode **gần nhất** cho mỗi block — nếu DataNode đó lỗi, tự động chuyển sang bản sao kế tiếp
- $\Rightarrow$ Không có single point of failure khi đọc: luôn còn ít nhất 2 bản sao dự phòng

# HDFS — tóm tắt các tham số cấu hình chính

| Tham số | Giá trị mặc định | Ý nghĩa |
|---|---|---|
| `dfs.blocksize` | 128 MB | Kích thước 1 block dữ liệu |
| `dfs.replication` | 3 | Số bản sao mỗi block |
| Heartbeat interval | 3 giây | Tần suất DataNode báo cáo NameNode |
| Block report interval | 6 giờ | Tần suất báo cáo toàn bộ danh sách block |
| Stale DataNode timeout | 10.5 phút | Thời gian không nhận heartbeat trước khi coi là chết |

# HDFS — ưu điểm & hạn chế

- Ưu điểm
  - Mở rộng theo chiều ngang gần như không giới hạn (thêm DataNode)
  - Chịu lỗi tốt nhờ replication, tự động phát hiện & phục hồi
  - Thông lượng đọc/ghi tuần tự rất cao — lý tưởng cho MapReduce/Spark
- Hạn chế
  - **Không phù hợp** cho truy cập ngẫu nhiên độ trễ thấp (không phải database)
  - Không tốt với **nhiều file nhỏ** — mỗi file/block chiếm metadata trên NameNode, dễ làm NameNode quá tải bộ nhớ
  - Triển khai & vận hành phức tạp hơn hẳn một ổ đĩa đơn hoặc dịch vụ cloud managed

# HDFS — vấn đề "nhiều file nhỏ" (Small Files Problem)

- NameNode giữ metadata của **mọi** file/block trong **RAM** — mỗi file, thư mục, block chiếm khoảng **150 byte** bộ nhớ
- *Ví dụ số*: 100 triệu file nhỏ $\Rightarrow \approx 100\text{triệu} \times 150\text{byte} = 15\text{GB}$ RAM chỉ để lưu metadata — chưa kể block report và cấu trúc phụ trợ khác
- Nếu dữ liệu gồm hàng chục triệu file **nhỏ hơn nhiều** so với block size 128MB (ví dụ hàng triệu ảnh vài KB), NameNode dễ **quá tải bộ nhớ** dù tổng dung lượng dữ liệu thực tế không lớn
- Giải pháp thường dùng: **gộp file nhỏ** thành file lớn hơn trước khi nạp vào HDFS (ví dụ dùng SequenceFile/HAR archive, hoặc đơn giản là ghi trực tiếp dưới dạng Parquet — sẽ học ở phần Định dạng file)

# HDFS — High Availability: khắc phục Single Point of Failure

- Vì NameNode là **single point of failure**, các phiên bản HDFS hiện đại hỗ trợ **HDFS High Availability (HA)**: chạy **2 NameNode** (Active/Standby) đồng bộ trạng thái liên tục qua một nhóm **JournalNode**
- Nếu Active NameNode chết, Standby NameNode **tự động chuyển sang Active** (failover) trong vài giây, không cần can thiệp thủ công
- **HDFS Federation** (mở rộng khác): cho phép **nhiều NameNode độc lập**, mỗi NameNode quản lý một phần namespace riêng — giúp mở rộng **giới hạn bộ nhớ** của một NameNode đơn lẻ khi cụm quá lớn

# HDFS 3.x — Erasure Coding: giảm chi phí lưu trữ so với replication

- Thay vì nhân bản **toàn bộ** dữ liệu 3 lần (overhead 200%), **Erasure Coding** chia dữ liệu thành các khối và tính thêm **khối chẵn lẻ** (parity) — giống nguyên lý RAID trong lưu trữ đĩa truyền thống
- Ví dụ chính sách phổ biến: $6$ khối dữ liệu $+3$ khối parity — chỉ cần bất kỳ $6$ trong $9$ khối là khôi phục được toàn bộ dữ liệu gốc, overhead khoảng $50\%$ thay vì $200\%$
- Đánh đổi: phục hồi dữ liệu khi có lỗi **tốn CPU hơn** (phải tính toán lại từ parity) so với chỉ copy nguyên một bản sao — vì vậy Erasure Coding thường dùng cho dữ liệu **ít truy cập, cần lưu lâu dài** (cold data), còn dữ liệu "nóng" vẫn ưu tiên replication truyền thống

# Từ HDFS đến Google File System



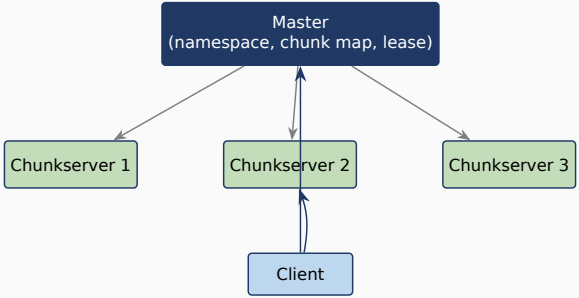

*HDFS không phải là ý tưởng hoàn toàn mới — nó được thiết kế **lấy cảm hứng trực tiếp** từ một hệ thống ra đời trước đó tại Google.*

- Google File System (GFS) — công bố năm 2003, trước cả Hadoop
- Rất nhiều khái niệm ta vừa học (block lớn, replication $= 3$, NameNode/DataNode) đều có **tương ứng gần như 1–1** trong GFS (Master/Chunkserver)
- $\rightarrow$ Cùng tìm hiểu bài báo gốc để hiểu **vì sao** các lựa chọn thiết kế này ra đời

# Google File System (GFS) — bối cảnh ra đời

- Bài báo: "The Google File System" — Sanjay Ghemawat, Howard Gobioff, Shun-Tak Leung, **SOSP 2003** (Symposium on Operating Systems Principles)
- Google cần lưu trữ & xử lý dữ liệu quy mô **chưa từng có** (crawl toàn bộ Web) — các hệ thống file truyền thống lúc đó không đáp ứng được
- Triết lý thiết kế táo bạo: **chấp nhận** phần cứng giá rẻ, dễ hỏng (commodity hardware), rồi xây dựng **độ tin cậy ở tầng phần mềm**
- Quy mô triển khai thực tế theo bài báo: **hơn 1.000 node lưu trữ, hơn 300TB dung lượng đĩa**, hàng trăm client truy cập liên tục — rất lớn vào năm 2003

# GFS — kiến trúc Master & Chunkserver




Client hỏi Master vị trí chunk, sau đó đọc/ghi dữ liệu **trực tiếp** với Chunkserver

*Client hỏi Master vị trí chunk, sau đó đọc/ghi dữ liệu trực tiếp với Chunkserver — cùng triết lý với NameNode/DataNode của HDFS*

# GFS — Master (tương đương NameNode)

- Quản lý **namespace**, quyền truy cập, và **ánh xạ file** $\rightarrow$ **chunk**
- **Không lưu vị trí chunk một cách bền vững (persistent)** — mỗi khi khởi động lại, Master **hỏi lại từng Chunkserver** xem nó đang giữ những chunk nào
- Chịu trách nhiệm các hoạt động toàn cục: **quản lý lease** (xem slide sau), **garbage collection** chunk mồ côi, **di chuyển chunk** giữa các Chunkserver để cân bằng tải
- Nguồn: *The Google File System*, SOSP 2003 (PDF chính thức, Google Research)

# GFS — Chunkserver (tương đương DataNode)

- Lưu trữ từng **chunk** như một file Linux thông thường trên đĩa cục bộ
- Xử lý các yêu cầu đọc/ghi theo **chunk handle** (định danh) và **khoảng byte** (byte range) do client chỉ định
- Tự duy trì **checksum** cho từng chunk để phát hiện dữ liệu bị hỏng (data corruption) — một cơ chế mà HDFS cũng kế thừa

# GFS — Chunk size: vì sao lớn hơn cả HDFS?

GFS chọn kích thước chunk:

$$\text{chunk size} = 64\text{MB}$$

Theo nguyên văn bài báo: "much larger than typical file system blocksizes" — mục tiêu là *giảm số lần client phải liên lạc với Master* và cho phép duy trì *kết nối TCP bền vững* (persistent) giữa client và chunkserver khi đọc/ghi nhiều dữ liệu liên tiếp.

Replication mặc định của GFS cũng là:

$$\text{số bản sao} = 3$$

"Người dùng có thể chỉ định mức replication khác nhau cho từng vùng của namespace" — linh hoạt hơn một cấu hình cố định toàn cục.

# GFS — Lease: ai được quyền ghi trước?

- Bài toán: nhiều client có thể muốn **ghi đồng thời** vào cùng một chunk — cần một cơ chế quyết định **thứ tự** ghi thống nhất
- Giải pháp: Master cấp một **lease** cho **một** bản sao (gọi là *primary*) — bản sao này có quyền **quyết định thứ tự** (assign serial number) cho các lượt ghi (mutation)
- Lease có **thời hạn khởi tạo 60 giây**, có thể được gia hạn vô thời hạn nếu chunk vẫn đang được ghi tích cực
- $\Rightarrow$ Master **không cần tham gia** vào từng lượt ghi cụ thể — chỉ cấp quyền một lần, giảm tải cho Master (giống lý do NameNode không nằm trên đường truyền dữ liệu của HDFS)

# GFS — mô hình nhất quán (Consistency Model)

- GFS **không** đảm bảo nhất quán mạnh (strong consistency) tuyệt đối như một database truyền thống — đây là đánh đổi có chủ đích để đạt hiệu năng cao
- Hai trạng thái then chốt sau một lượt ghi (mutation):
  - **Defined** (xác định): mọi client đều thấy đúng những gì lượt ghi đó đã ghi
  - **Consistent nhưng undefined** (nhất quán nhưng không xác định): mọi client thấy **cùng một dữ liệu**, nhưng dữ liệu đó có thể là **pha trộn** từ nhiều lượt ghi đồng thời thành công
- Bài học thiết kế: hệ thống phân tán quy mô lớn thường phải **đánh đổi** giữa nhất quán tuyệt đối và hiệu năng — chủ đề này sẽ quay lại ở Phần 5 (NoSQL, định lý CAP)

# GFS — Heartbeat & phát hiện lỗi

- Master và Chunkserver trao đổi **heartbeat** định kỳ — Master gửi chỉ thị (instructions), Chunkserver báo cáo trạng thái
- Thông qua heartbeat, Master gia hạn lease, thu thập trạng thái chunk, và **phát hiện Chunkserver bị chết** (không phản hồi)
- Khi phát hiện mất một bản sao, Master lên lịch **tạo lại bản sao mới** trên một Chunkserver khác — tương tự cơ chế re-replication của HDFS

# Số liệu thật: hai cụm GFS đang chạy tại Google (2003)

| Đặc điểm | Cluster A | Cluster B |
|---|---|---|
| Số Chunkserver | 342 | 227 |
| Dung lượng đĩa khả dụng | 72 TB | 180 TB |
| Dung lượng đĩa đã dùng | 55 TB | 155 TB |
| Số file | 735 nghìn | 737 nghìn |
| Số file "chết" (dead) | 22 nghìn | 232 nghìn |
| Số chunk | 992 nghìn | 1.550 nghìn |
| Metadata tại Chunkserver | 13 GB | 21 GB |
| **Metadata tại Master** | **48 MB** | **60 MB** |

# Vì sao bảng số liệu thật này quan trọng?

- Nguồn: Table 2, Ghemawat, Gobioff, Leung — *The Google File System* (SOSP 2003) — đo trên 2 cụm GFS **đang chạy thật** tại Google, không phải số liệu mô phỏng
- Điểm đáng chú ý nhất: dù quản lý **hàng trăm Chunkserver** và **hàng chục/trăm TB dữ liệu**, Master chỉ giữ **48–60 MB metadata** — xác nhận bằng số liệu thật cho thiết kế "giữ toàn bộ metadata trong RAM của Master" đã học
- Cluster A (nghiên cứu, do người dùng) vs. Cluster B (production, tự động): B có **tỷ lệ file chết cao hơn hẳn** (232k/737k so với 22k/735k) — vì các tác vụ production tạo/xóa file liên tục, minh chứng thêm cho cơ chế garbage collection của Master

# So sánh trực tiếp: GFS vs. HDFS

| Khái niệm | GFS (Google, 2003) | HDFS (Hadoop) |
|---|---|---|
| Node quản lý metadata | Master | NameNode |
| Node lưu dữ liệu | Chunkserver | DataNode |
| Đơn vị chia nhỏ file | Chunk (64MB) | Block (128MB) |
| Replication mặc định | 3 | 3 |
| Cơ chế phát hiện lỗi | Heartbeat | Heartbeat |
| Mô hình nhất quán | Defined / Consistent | Write-once-read-many |

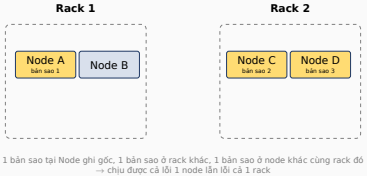
# GFS — vì sao đây là bài báo “phải đọc” của ngành?

- GFS là một trong những công trình **đặt nền móng** cho toàn bộ làn sóng Big Data hiện đại: HDFS (Hadoop), Colossus (thế hệ kế tiếp của GFS tại Google), và gián tiếp truyền cảm hứng cho nhiều hệ NoSQL
- Bài học kiến trúc cốt lõi vẫn còn nguyên giá trị: **tách compute khỏi phần điều phối metadata**, **chấp nhận lỗi là điều tất yếu** thay vì cố ngăn chặn hoàn toàn, và **đơn giản hóa mô hình nhất quán** để đổi lấy hiệu năng
- Đọc bài báo gốc giúp hiểu **lý do thiết kế** (design rationale) đằng sau các con số “128MB”, “replication=3” mà ta thường chỉ học thuộc lòng khi dùng Hadoop

# Chịu lỗi trong lưu trữ phân tán — vì sao cần thiết kế kỹ?

- Với hàng nghìn máy, xác suất **ít nhất một máy hỏng tại một thời điểm bất kỳ** là rất cao — đây là hệ quả trực tiếp của giả định "hardware failure is the norm" đã học
- Ba câu hỏi thiết kế cốt lõi: (1) đặt bản sao **ở đâu** để giảm thiểu rủi ro mất dữ liệu? (2) làm sao **phát hiện** một node đã chết? (3) làm sao **phục hồi** khi có node chết?

# Replica Placement — chính sách đặt bản sao theo Rack




*Chính sách rack-aware: 1 bản sao tại node ghi gốc, 1 bản sao ở rack khác, 1 bản sao ở node khác cùng rack đó*

## Định nghĩa nhanh: Node & Rack

- **Node**: một **máy chủ vật lý** trong cụm — lưu dữ liệu (DataNode/Chunkserver) hoặc quản lý (NameNode/Master), mỗi node có CPU/RAM/đĩa riêng.
- **Rack**: một **tủ máy chủ** chứa nhiều node, dùng **chung nguồn điện** & **chung một switch mạng** (Top-of-Rack switch) — nhiều rack hợp thành một cụm (cluster) trong trung tâm dữ liệu.
- $\Rightarrow$ Node **cùng rack** chia sẻ điểm lỗi chung (nguồn/switch) — vì vậy chính sách đặt bản sao phải **tính đến rack**, không chỉ tính đến node.

# Vì sao đặt bản sao theo Rack (rack-aware placement)?

- Một **rack** là một tủ chứa nhiều máy chủ, dùng chung nguồn điện & switch mạng — nếu rack đó mất điện/hỏng switch, **toàn bộ máy trong rack** mất kết nối cùng lúc
- Nếu đặt cả 3 bản sao trong **cùng một rack**: nhanh (băng thông nội rack cao) nhưng rủi ro — một sự cố rack là mất **toàn bộ** dữ liệu
- Nếu đặt cả 3 bản sao ở **3 rack khác nhau**: an toàn tối đa nhưng chậm — ghi phải đi qua nhiều switch liên-rack (tốn băng thông backbone)
- Chính sách HDFS/GFS chọn điểm cân bằng: **1 bản tại chỗ + 1 bản rack khác + 1 bản node khác cùng rack đó** — chịu được cả lỗi 1 node lẫn lỗi nguyên 1 rack, mà vẫn tiết kiệm băng thông liên-rack

# Heartbeat & phát hiện lỗi node

- Mỗi DataNode/Chunkserver gửi **heartbeat** định kỳ (HDFS: mặc định 3 giây/lần) về Master/NameNode để báo "tôi vẫn còn sống"
- Nếu Master **không nhận được heartbeat** trong một khoảng thời gian nhất định (HDFS: khoảng 10.5 phút mặc định), node đó bị đánh dấu **chết** (dead/stale)
- Khi một node bị đánh dấu chết, mọi block/chunk nó từng giữ ngay lập tức bị coi là **under-replicated** (thiếu bản sao) — kích hoạt quy trình phục hồi tự động

# Re-replication — tự phục hồi sau lỗi

- Master/NameNode duy trì một **hàng đợi ưu tiên** các block đang thiếu bản sao (under-replicated)
- Với mỗi block thiếu bản sao, Master chọn một DataNode còn sống **giữ sẵn 1 bản sao** làm nguồn, và một DataNode khác (còn dung lượng trống) làm đích, ra lệnh sao chép trực tiếp **giữa hai DataNode** (không đi qua Master)
- Quá trình này diễn ra **hoàn toàn tự động**, không cần can thiệp thủ công — đây chính là ý nghĩa của "tự phục hồi" (self-healing) trong định nghĩa gốc của HDFS
- Ngược lại, nếu một block có **quá nhiều** bản sao (over-replicated — ví dụ sau khi một node từng "chết" rồi quay lại), Master ra lệnh **xóa bớt** bản sao thừa để tiết kiệm dung lượng

# Checksum — phát hiện dữ liệu bị hỏng âm thầm

- Checksum là gì?
  - Một **giá trị nhỏ, kích thước cố định** (ví dụ 4 byte) được **tính toán từ nội dung dữ liệu** bằng một hàm toán học — giống tinh thần **hash function** đã học ở Tuần 2, nhưng mục tiêu là **phát hiện lỗi**, không phải phân bố dữ liệu
  - Đặc tính cốt lõi: chỉ cần **1 bit dữ liệu gốc thay đổi** (dù rất nhỏ, do bit-rot hay lỗi đĩa), checksum tính lại **gần như chắc chắn sẽ khác** — nhờ đó phát hiện được cả những hỏng hóc rất nhỏ
- Một node có thể **không chết** nhưng dữ liệu trên đĩa của nó bị **hỏng âm thầm** (silent corruption) — do lỗi đĩa, bit-rot, hoặc lỗi phần cứng khác — nguy hiểm hơn lỗi "chết hẳn" vì khó phát hiện
- Giải pháp: mỗi DataNode/Chunkserver lưu kèm **checksum** cho từng block dữ liệu (HDFS dùng CRC32C theo từng đoạn 512 byte)
- Khi đọc dữ liệu, checksum được **tính lại và so sánh** — nếu không khớp, block đó bị đánh dấu hỏng, hệ thống tự động đọc từ **bản sao khác** và kích hoạt tạo lại bản sao thay thế

# Ví dụ số đơn giản: checksum hoạt động ra sao?

Dùng một hàm checksum **đơn giản hóa** để minh họa (thực tế HDFS dùng CRC32C phức tạp hơn nhiều, nhưng nguyên lý giống hệt): **tổng mã ASCII các ký tự, lấy phần dư cho 256**.

**Dữ liệu gốc**: chuỗi "AB" $\rightarrow$ mã ASCII $65 + 66 = 131$

$$\text{checksum}_{\text{gốc}} = 131 \bmod 256 = 131$$

**Sau khi đĩa hỏng 1 bit**: "AB" bị đọc nhầm thành "AC" (chỉ 1 ký tự lệch 1 đơn vị) $\rightarrow$ mã ASCII $65 + 67 = 132$

$$\text{checksum}_{\text{đọc lại}} = 132 \bmod 256 = 132$$

$131 \neq 132 \Rightarrow$ **phát hiện ngay** dữ liệu đã bị hỏng, dù chỉ lệch đúng 1 ký tự — hệ thống lập tức chuyển sang đọc từ bản sao khác thay vì trả về dữ liệu sai cho người dùng.

# Vì sao checksum quan trọng — không thể chỉ dựa vào replication?

- **Replication** (đã học) giải quyết bài toán "node chết hẳn" — nhưng **không** tự phát hiện được trường hợp node vẫn sống, vẫn trả lời được, nhưng **trả về dữ liệu sai**
- Nếu không có checksum: hệ thống có thể **âm thầm phục vụ dữ liệu đã hỏng** cho người dùng trong thời gian dài mà không ai biết — hậu quả nghiêm trọng hơn nhiều so với việc mất hẳn 1 bản sao (trường hợp đó còn 2 bản sao khác bù vào)
- $\Rightarrow$ Checksum và Replication **bổ sung cho nhau**, không thay thế nhau: Replication đảm bảo **luôn còn dữ liệu để đọc**, Checksum đảm bảo dữ liệu đọc được đó **đúng là dữ liệu gốc**
- Đây là lý do cả GFS lẫn HDFS đều bắt buộc dùng checksum cho **mọi** block/chunk, dù đã có replication factor $= 3$ — hai cơ chế nhắm vào hai loại lỗi hoàn toàn khác nhau

# Ví dụ số: xác suất mất dữ liệu giảm ra sao nhờ replication?

Giả sử xác suất một node **bất kỳ hỏng trong một ngày** là $p = 0.001$ (0.1%) — một con số điển hình cho ổ đĩa cơ.

Với **1 bản sao** duy nhất, xác suất mất block đó trong ngày chính là $p = 0.001$.

Với **3 bản sao độc lập** (đặt trên 3 node khác nhau), xác suất *mất cả ba cùng lúc* trong một ngày:

$$p^3 = (0.001)^3 = 10^{-9}$$

Giảm từ $10^{-3}$ xuống $10^{-9}$ — tức **giảm rủi ro đi một triệu lần**. Đây chính là lý do định lượng vì sao replication factor $= 3$ mang lại độ tin cậy vượt trội, dù chi phí lưu trữ tăng gấp 3 lần (đã tính ở slide HDFS).

# Tổng kết: ba cơ chế chịu lỗi cốt lõi

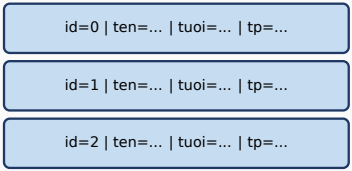
- **Replica placement** theo rack — giảm thiểu rủi ro mất dữ liệu do lỗi tương quan (correlated failure) trong cùng một rack
- **Heartbeat + re-replication** — phát hiện lỗi node và tự động phục hồi số lượng bản sao mà không cần can thiệp thủ công
- **Checksum** — phát hiện lỗi "âm thầm" mà heartbeat không thể phát hiện (node vẫn sống nhưng dữ liệu hỏng)
- Cả ba cơ chế này lặp lại (với biến thể riêng) trong hầu hết hệ thống lưu trữ phân tán hiện đại: HDFS, GFS, Cassandra, S3 nội bộ của AWS...

# Định dạng file — lưu "ở đâu" chưa đủ, còn phải lưu "như thế nào"

- HDFS/GFS quyết định **dữ liệu vật lý nằm ở đâu**, nhưng **cách sắp xếp bytes bên trong file** cũng ảnh hưởng cực lớn đến hiệu năng truy vấn
- Câu hỏi cốt lõi: khi lưu một bảng dữ liệu (nhiều dòng, nhiều cột), nên sắp xếp **theo dòng** (row-based) hay **theo cột** (columnar)?

# Row-based vs. Columnar Storage

## Row-based (CSV, Avro)




Đọc 1 dòng: nhanh
Đọc 1 cột trên nhiều dòng: chậm

## Columnar (Parquet, ORC)




Chỉ đọc cột cần (column pruning)
Nén tốt hơn (giá trị cùng cột giống kiểu)

*Row-based lưu liên tiếp từng dòng đầy đủ; Columnar lưu liên tiếp từng cột riêng biệt*

# Row-based storage (CSV, Avro, JSON Lines)

- Mỗi dòng (record) được lưu **liên tiếp, đầy đủ mọi cột** trên đĩa
- **Tối ưu khi**: cần đọc/ghi **toàn bộ một dòng** cùng lúc (ví dụ: hệ giao dịch OLTP — thêm/sửa/xóa một bản ghi khách hàng)
- **Kém hiệu quả khi**: chỉ cần **một vài cột** trên **rất nhiều dòng** — vẫn phải đọc toàn bộ dòng (kể cả các cột không cần) rồi mới lọc ra cột cần thiết
- Avro: định dạng row-based phổ biến trong hệ sinh thái Hadoop, có kèm **schema** (khác CSV thuần văn bản không có schema)

# Columnar storage (Parquet, ORC)

- Giá trị của **cùng một cột**, từ mọi dòng, được lưu **liên tiếp nhau** trên đĩa
- **Column pruning**: truy vấn chỉ cần đọc **đúng những cột được yêu cầu** (ví dụ `SELECT ten, tuoi FROM ...`) — bỏ qua hoàn toàn các cột khác, giảm I/O đáng kể
- **Predicate pushdown**: mỗi khối cột lưu kèm thống kê (min/max) — engine truy vấn có thể **bỏ qua cả khối** nếu biết chắc không khớp điều kiện lọc, mà không cần đọc dữ liệu
- **Nén tốt hơn**: giá trị trong cùng một cột thường **cùng kiểu, giá trị gần giống nhau** (ví dụ cột `tinh_thanh` chỉ có vài chục giá trị lặp lại) — các thuật toán nén (dictionary encoding, run-length encoding) đạt tỉ lệ nén cao hơn hẳn so với nén theo dòng
- Nguồn: Apache Parquet — tài liệu chính thức (parquet.apache.org)

# Ví dụ số: column pruning tiết kiệm I/O bao nhiêu?

Giả sử một bảng có **20 cột**, kích thước xấp xỉ đều nhau, tổng dung lượng file là **2GB**.

Truy vấn chỉ cần **2 cột** (ví dụ tính trung bình một chỉ số theo thời gian):

$$\text{với row-based (CSV/Avro)}: \quad \text{phải đọc toàn bộ } 2\text{GB}$$

$$\text{với columnar (Parquet)}: \quad \text{chỉ đọc} \approx \frac{2}{20} \times 2\text{GB} = 0.2\text{GB}$$

Giảm khối lượng I/O đi **10 lần** chỉ nhờ đổi định dạng lưu trữ — chưa tính thêm lợi ích từ nén tốt hơn và predicate pushdown. Đây là lý do gần như **mọi hệ Lakehouse hiện đại** (Delta Lake, Iceberg, Hudi — sẽ học ở các slide sau) đều xây dựng trên nền Parquet.

# ORC (Optimized Row Columnar) — điểm khác biệt với Parquet

- Cũng là định dạng columnar, ra đời trong hệ sinh thái **Hive** (kho dữ liệu trên Hadoop), tối ưu mạnh cho truy vấn kiểu SQL
- Có cấu trúc **Index nhẹ tích hợp sẵn** (lightweight indexes) giúp bỏ qua nhanh các nhóm dòng không khớp điều kiện
- Trong thực tế: **Parquet** phổ biến hơn trong hệ sinh thái Spark/Python/đa ngôn ngữ; **ORC** phổ biến hơn trong hệ sinh thái Hive thuần — cả hai đều là lựa chọn tốt cho Lakehouse

# Nén dữ liệu (Compression) — một chiều tối ưu song song

- Các định dạng columnar thường hỗ trợ nhiều thuật toán nén: **Snappy** (nén/giải nén nhanh, tỉ lệ nén vừa phải — lựa chọn mặc định phổ biến), **Gzip** (tỉ lệ nén cao hơn nhưng chậm hơn), **Zstandard (zstd)** (cân bằng tốc độ/tỉ lệ nén, ngày càng phổ biến)
- Nén giúp **giảm dung lượng lưu trữ** (tiết kiệm chi phí) **và giảm I/O khi đọc** (đọc ít byte hơn từ đĩa/mạng) — đổi lại chi phí CPU để giải nén
- Với dữ liệu columnar, nén theo cột thường đạt tỉ lệ nén **cao hơn hẳn** nén theo dòng, vì dữ liệu cùng cột có tính "đồng nhất" cao

# So sánh tổng hợp các định dạng file phổ biến

| Định dạng | Kiểu lưu trữ | Điểm mạnh chính |
|---|---|---|
| CSV | Row-based, không schema | Đơn giản, dễ đọc bằng mắt người, tương thích rộng |
| Avro | Row-based, có schema | Tốt cho ghi liên tục (streaming), schema evolution |
| Parquet | Columnar, có schema | Truy vấn phân tích (OLAP), nén tốt, chuẩn Lakehouse |
| ORC | Columnar, có schema | Tối ưu cho Hive, index nhẹ tích hợp |

# Partitioning — tổ chức file theo thư mục để tăng tốc truy vấn

- Ngoài định dạng file, cách **tổ chức thư mục** cũng ảnh hưởng lớn đến hiệu năng — kỹ thuật phổ biến nhất: **Hive-style partitioning**, chia dữ liệu thành thư mục con theo giá trị một/vài cột
- *Ví dụ*: `data/nam=2026/thang=08/ngay=23/*.parquet` — mỗi tổ hợp (năm, tháng, ngày) là một thư mục riêng
- Truy vấn có điều kiện `WHERE nam=2026 AND thang=08` chỉ cần **quét đúng thư mục liên quan** (*partition pruning*) — bỏ qua hoàn toàn phần còn lại, kết hợp với column pruning của Parquet cho hiệu quả nhân đôi
- Rủi ro: chọn cột partition có **quá nhiều giá trị khác nhau** (ví dụ partition theo `user_id`) sẽ tạo ra **hàng triệu thư mục nhỏ** — lại rơi vào đúng vấn đề "nhiều file nhỏ" đã học ở phần HDFS

# Ví dụ số: hiệu quả nén thực tế của Parquet

Số liệu benchmark điển hình được nhiều báo cáo kỹ thuật ghi nhận khi chuyển từ CSV sang Parquet+Snappy trên dữ liệu dạng bảng (nhiều cột lặp lại giá trị):

| **Định dạng** | **Dung lượng tương đối** | **Tốc độ đọc 1 cột** |
|---|---|---|
| CSV (không nén) | $100\%$ (nền tham chiếu) | chậm nhất |
| CSV + Gzip | $\approx 20$–$30\%$ | vẫn phải đọc toàn dòng |
| Parquet + Snappy | $\approx 15$–$25\%$ | nhanh hơn $5$–$10$ lần |

Con số cụ thể luôn phụ thuộc vào đặc điểm dữ liệu thực tế (độ lặp lại, số cột, kiểu dữ liệu) — phần thực hành sẽ đo **trực tiếp trên dữ liệu do các bạn tạo ra** thay vì chỉ tin vào số liệu tham khảo.

# Định dạng file — khi nào chọn cái nào?

- **Dữ liệu đang được ghi liên tục, đọc theo dòng** (log hệ thống, sự kiện streaming): ưu tiên **Avro** hoặc JSON Lines
- **Dữ liệu dùng cho phân tích, báo cáo, truy vấn tổng hợp** (OLAP, dashboard, huấn luyện ML): ưu tiên **Parquet** hoặc ORC
- **Dữ liệu trao đổi với người dùng cuối/hệ thống khác** không kiểm soát được: CSV vẫn phổ biến vì tính tương thích, dù kém hiệu quả nhất
- → Trong thực hành cuối phần này, ta sẽ **đo trực tiếp bằng Python** sự khác biệt giữa CSV và Parquet trên cùng một tập dữ liệu

# Từ tự vận hành cụm HDFS sang thuê lưu trữ trên Cloud

- Vận hành một cụm HDFS đòi hỏi **tự quản lý phần cứng**: mua máy, lắp rack, thay ổ đĩa hỏng, nâng cấp phần mềm — chi phí vận hành (operational cost) rất lớn
- Từ khoảng 2010s trở đi, phần lớn tổ chức chuyển sang dùng **dịch vụ lưu trữ đối tượng** (object storage) do các nhà cung cấp Cloud quản lý: Amazon S3, Google Cloud Storage (GCS), Azure Blob Storage
- Câu hỏi: object storage khác gì HDFS về mặt kiến trúc và khi nào nên dùng cái nào?

# HDFS (block storage) vs. Object Storage (Cloud)

**HDFS (block storage)**



Cluster máy chủ tự quản lý
CPU đi kèm với đĩa lưu trữ

**Object Storage (S3, GCS)**



Dịch vụ managed, tách rời
compute và storage

*HDFS: cluster tự quản lý, compute và storage đi cùng nhau trên từng máy. Object Storage: dịch vụ managed, tách rời compute khỏi storage*

# Object Storage — mô hình dữ liệu khác biệt

- Không có khái niệm "block" hay "thư mục thật" như HDFS — dữ liệu được lưu dưới dạng **object** (đối tượng) trong một **bucket** (thùng chứa phẳng)
- Mỗi object có một **key** (giống như đường dẫn, ví dụ `s3://bucket/nam=2026/thang=08/data.parquet`) — nhưng thực chất không có cây thư mục thật, chỉ là **tên phẳng** có dấu `/`
- Truy cập qua **API HTTP(S)** (REST) thay vì giao thức file system truyền thống — linh hoạt nhưng độ trễ mỗi request thường cao hơn truy cập file cục bộ

# Tách rời Compute và Storage — thay đổi kiến trúc quan trọng nhất

- Với HDFS: máy vừa **lưu dữ liệu** vừa **chạy tính toán** (DataNode + worker MapReduce/Spark trên cùng một máy) — tận dụng nguyên lý "di chuyển tính toán rẻ hơn di chuyển dữ liệu" đã học
- Với Object Storage: **lưu trữ** (S3/GCS) và **tính toán** (cụm Spark/EMR/Databricks) là hai dịch vụ **độc lập**, có thể scale riêng — tắt cụm tính toán mà không mất dữ liệu, hoặc scale cụm tính toán mà không cần dời dữ liệu
- Đánh đổi: mất lợi thế "data locality" (đọc dữ liệu qua mạng thay vì đĩa cục bộ) — bù lại bằng chi phí vận hành thấp hơn nhiều và tính đàn hồi (elasticity) cao hơn

# So sánh HDFS vs. Object Storage (S3/GCS/Azure Blob)

| Tiêu chí | HDFS | Object Storage (Cloud) |
|---|---|---|
| Mô hình vận hành | Tự quản lý cụm máy chủ | Dịch vụ managed, trả tiền theo dùng |
| Compute & Storage | Gắn liền nhau trên từng máy | Tách rời hoàn toàn |
| Độ bền dữ liệu | Phụ thuộc cấu hình replication | Rất cao (thường 99.999999999%) |
| Chi phí vận hành | Cao (tự bảo trì phần cứng) | Thấp (nhà cung cấp quản lý) |
| Độ trễ truy cập | Thấp hơn (đĩa cục bộ/mạng nội bộ) | Cao hơn (API qua Internet/VPC) |

# S3 — các storage class (tầng lưu trữ) theo chi phí/tần suất truy cập

| Storage class | Trường hợp dùng | Đặc điểm chi phí |
|---|---|---|
| S3 Standard | Dữ liệu truy cập thường xuyên | Chi phí lưu trữ cao nhất, không phí truy xuất |
| S3 Infrequent Access | Truy cập không thường xuyên nhưng cần nhanh khi cần | Lưu trữ rẻ hơn, có phí mỗi lần đọc |
| S3 Glacier | Lưu trữ lâu dài, hiếm khi truy cập (back-up/archive) | Rẻ nhất, thời gian phục hồi từ phút đến giờ |

# Ý nghĩa của storage class cho kiến trúc Lakehouse

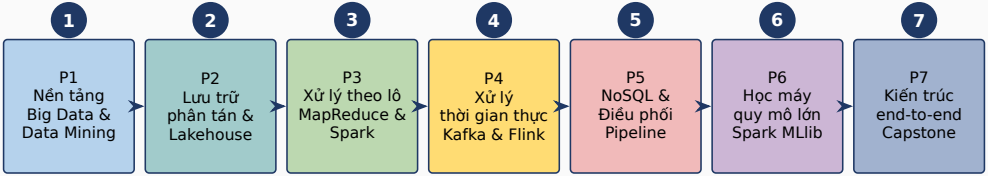
- Lakehouse hiện đại thường áp dụng **policy vòng đời dữ liệu** (lifecycle policy): dữ liệu mới/hay dùng → Standard; dữ liệu cũ, ít truy vấn → tự động chuyển sang tầng rẻ hơn sau N ngày
- Đây là một lợi thế **tách compute khỏi storage** mang lại: tối ưu **chi phí theo tuổi dữ liệu** mà không cần di chuyển dữ liệu thủ công hay thay đổi hệ thống truy vấn phía trên
- Về mặt **consistency**: các dịch vụ object storage hiện đại (S3 từ 2020) đã hỗ trợ **strong read-after-write consistency** — đọc ngay sau khi ghi luôn thấy dữ liệu mới nhất, khác với mô hình "eventual consistency" từng phổ biến trước đây

# Vậy HDFS còn được dùng không?

- Vẫn được dùng rộng rãi trong các **trung tâm dữ liệu tự quản lý (on-premise)** — ngân hàng, tổ chức chính phủ, hoặc nơi có yêu cầu nghiêm ngặt về vị trí dữ liệu (data residency)
- Nhiều tổ chức dùng mô hình **lai (hybrid)**: object storage làm nơi lưu trữ chính ("data lake"), một cụm HDFS/Spark nhỏ hơn xử lý các workload cần độ trễ thấp
- Xu hướng hiện nay: hầu hết nền tảng Lakehouse mới (Delta Lake, Iceberg — sẽ học tiếp) đều thiết kế để chạy **trên object storage** thay vì HDFS, chính vì lợi ích tách compute/storage

# DATA WAREHOUSE, DATA LAKE VÀ LAKEHOUSE

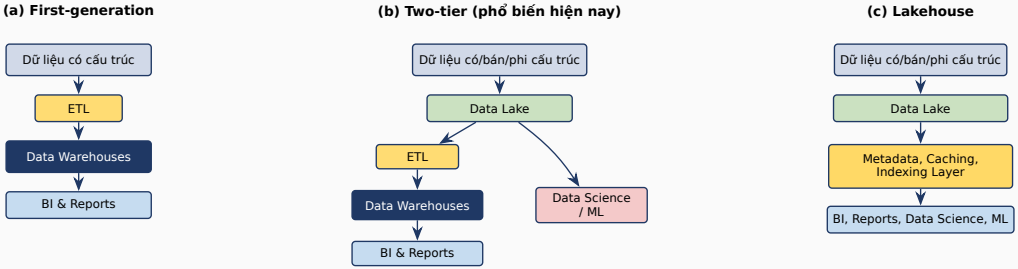
*Ba mô hình tổ chức dữ liệu ở tầng cao hơn HDFS/Object Storage*



# Nhắc lại nhanh từ Tuần 2

- Ở Tuần 2, ta đã giới thiệu **sơ lược** ba khái niệm Data Warehouse, Data Lake, Lakehouse như một phần của bức tranh hệ sinh thái Big Data hiện đại
- Ở đây, ta đi **sâu hơn**: vì sao mỗi mô hình ra đời, hạn chế của mô hình trước dẫn đến mô hình sau như thế nào, và các công nghệ cụ thể hiện thực hóa Lakehouse

# Ba thế hệ kiến trúc nền tảng dữ liệu (theo bài báo Lakehouse)




Nguồn: Figure 1, Armbrust et al. — *Lakehouse: A New Generation of Open Systems* (CIDR 2021)
Lakehouse gộp lớp quản trị (metadata/caching/indexing) ngay trên Data Lake — không cần ETL sang một kho riêng

*Nguồn: Figure 1, Armbrust et al. — CIDR 2021. Ba slide tiếp theo sẽ đi sâu vào từng thế hệ (a), (b), (c)*

# Data Warehouse — điểm khởi đầu (từ thập niên 1980–1990)

- Lưu trữ dữ liệu **đã qua xử lý, có cấu trúc chặt chẽ** (structured), theo mô hình quan hệ (bảng, cột, kiểu dữ liệu cố định)
- **Schema-on-write**: phải định nghĩa lược đồ (schema) **trước khi** nạp dữ liệu vào — dữ liệu không khớp schema sẽ bị từ chối ngay từ bước ETL
- Ưu điểm: chất lượng dữ liệu cao, truy vấn SQL nhanh, phù hợp báo cáo/BI truyền thống
- Hạn chế: **chi phí lưu trữ cao** (phần cứng chuyên dụng, định dạng độc quyền), **cứng nhắc** với dữ liệu phi cấu trúc (ảnh, video, log tự do) và khối lượng cực lớn

# Data Lake — giải pháp cho hạn chế của Data Warehouse (2010s)

- Ý tưởng: lưu **mọi loại dữ liệu** (có cấu trúc, bán cấu trúc, phi cấu trúc) ở dạng **thô, nguyên bản** trên nền tảng lưu trữ giá rẻ (HDFS ban đầu, sau này là object storage)
- **Schema-on-read**: **không** cần định nghĩa schema trước — schema chỉ được áp dụng **khi đọc** dữ liệu ra để xử lý, cho phép lưu trữ cực kỳ linh hoạt
- Ưu điểm: chi phí lưu trữ rẻ hơn nhiều, linh hoạt với mọi định dạng, phù hợp cả khối lượng dữ liệu khổng lồ
- Hạn chế nghiêm trọng: **thiếu quản trị** (governance) — không có ACID transaction, không kiểm soát chất lượng, dễ trở thành "data swamp" (đầm lầy dữ liệu, xem slide sau)

# Data Swamp — khi Data Lake đi sai hướng

- "Data swamp" là thuật ngữ mô tả một Data Lake đã **mất kiểm soát**: dữ liệu chồng chất không có metadata, không ai biết nguồn gốc/chất lượng, không thể truy vấn tin cậy
- Nguyên nhân điển hình: (1) không có **catalog/metadata** theo dõi dữ liệu gì đang có, (2) không có **transaction** — ghi dở dang do lỗi để lại dữ liệu hỏng nửa chừng, (3) không **kiểm soát version/schema evolution** — dữ liệu cũ và mới không tương thích nhau
- Đây chính là động lực trực tiếp dẫn đến sự ra đời của **Lakehouse** — thêm lớp quản trị lên trên Data Lake mà không đánh mất tính linh hoạt & chi phí thấp của nó

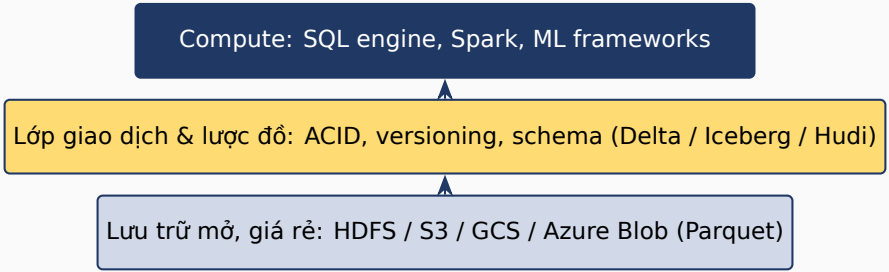
# Lakehouse — định nghĩa chính thức

Theo bài báo *"Lakehouse: A New Generation of Open Source Systems and Applications"* (Armbrust et al., **CIDR 2021**):

> *"Một lakehouse là hệ thống quản lý dữ liệu dựa trên lưu trữ* ***chi phí thấp và truy cập trực tiếp được****, đồng thời cung cấp các tính năng quản lý DBMS phân tích truyền thống như* ***ACID transaction, versioning dữ liệu, kiểm toán (auditing), lập chỉ mục (indexing), caching, và tối ưu hóa truy vấn****."*

Nói ngắn gọn: Lakehouse = **Data Lake** (lưu trữ mở, rẻ) + **lớp quản trị giao dịch** vốn chỉ có ở Data Warehouse.

# Kiến trúc phân lớp của Lakehouse




Lakehouse = Data Lake (lưu trữ mở, rẻ) + lớp quản lý giao dịch của Data Warehouse

*Lớp giao dịch & lược đồ (Delta/Iceberg/Hudi) nằm giữa lớp lưu trữ mở giá rẻ và lớp compute — mang lại cả hai ưu điểm*

# Ba mục tiêu thiết kế của Lakehouse (theo bài báo Armbrust et al.)

- 1. Định dạng dữ liệu mở (Open formats)
  - Xây dựng trên các định dạng chuẩn mở như **Parquet, ORC** — không khóa vào định dạng độc quyền của một nhà cung cấp
- 2. Hỗ trợ cả BI & Machine Learning
  - Không chỉ phục vụ truy vấn SQL truyền thống, mà còn là **nguồn dữ liệu trực tiếp** cho huấn luyện mô hình ML/AI (liên hệ Phần 6)
- 3. Hiệu năng cạnh tranh với Data Warehouse
  - Đạt tốc độ truy vấn tương đương các kho dữ liệu cloud truyền thống nhờ caching, index phụ trợ, và tối ưu bố cục dữ liệu (data layout)

# Số liệu thật: Lakehouse có thực sự nhanh & rẻ như DW không?

| Hệ thống | Thời gian chạy 99 truy vấn TPC-DS | Chi phí (USD) |
|---|---|---|
| DW1 (cloud DW phổ biến) | 2.996 s | $153 |
| DW2 | 7.143 s | $286 |
| DW3 | 5.793 s | $206 |
| DW4 | 37.283 s | $570 |
| **Delta Engine (on-demand)** | **3.302 s** | **$104** |
| **Delta Engine (spot)** | **3.252 s** | **$56** |

# Đọc bảng benchmark TPC-DS trên như thế nào?

- Nguồn: Figure 3, Armbrust et al. (CIDR 2021) — đo trên bộ benchmark chuẩn công nghiệp **TPC-DS ở scale factor 30.000** (30 nghìn tỷ dòng dữ liệu), chạy cùng cấu hình phần cứng (960 vCPU) trên AWS/Azure/GCS cho cả 4 data warehouse (ẩn danh DW1–DW4) lẫn Delta Engine (Lakehouse)
- Kết quả: Delta Engine (chạy trên Lakehouse, dữ liệu ở định dạng Parquet mở) đạt tốc độ **tương đương hoặc nhanh hơn** 3/4 data warehouse thương mại đóng — trong khi chi phí **thấp hơn cả 4**
- Đây chính là **bằng chứng thực nghiệm** cho mục tiêu thiết kế thứ 3 (hiệu năng cạnh tranh) vừa nêu ở slide trước — không chỉ là tuyên bố lý thuyết

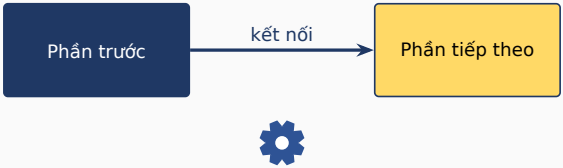
# So sánh tổng hợp: Data Warehouse vs. Data Lake vs. Lakehouse

| Tiêu chí | Data Warehouse | Data Lake | Lakehouse |
|---|---|---|---|
| Schema | Schema-on-write | Schema-on-read | Cả hai (linh hoạt) |
| ACID transaction | Có | Không | Có |
| Chi phí lưu trữ | Cao | Thấp | Thấp |
| Dữ liệu phi cấu trúc | Kém | Tốt | Tốt |
| Hỗ trợ ML trực tiếp | Hạn chế | Có (nhưng thiếu quản trị) | Có, kèm quản trị đầy đủ |

# Câu hỏi ôn nhanh: phân biệt Schema-on-write vs Schema-on-read

- **Schema-on-write** (Data Warehouse): kiểm tra dữ liệu **trước khi** ghi — lỗi bị chặn ngay từ đầu, nhưng **chậm** khi nạp dữ liệu và **cứng nhắc** với dữ liệu chưa biết cấu trúc trước
- **Schema-on-read** (Data Lake thuần): ghi trước, kiểm tra **khi đọc** — nạp dữ liệu **nhanh, linh hoạt**, nhưng dễ phát hiện lỗi **quá muộn** (khi đã có hàng nghìn bản ghi hỏng)
- Lakehouse dung hòa: cho phép ghi linh hoạt như Data Lake, nhưng có **schema enforcement tùy chọn** (đã học ở Delta Lake) — bật lên khi cần độ tin cậy cao, tắt đi khi cần tốc độ nạp dữ liệu

# Vậy công nghệ nào hiện thực hóa Lakehouse?



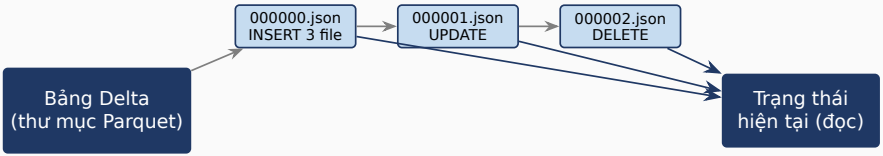

*Bản thân khái niệm "Lakehouse" chỉ là một **kiến trúc** — cần có công nghệ cụ thể để hiện thực hóa **lớp giao dịch & lược đồ** trên nền Parquet/object storage.*

- Ba công nghệ **table format** phổ biến nhất hiện nay: **Delta Lake** (Databricks), **Apache Iceberg** (Netflix, nay là Apache), **Apache Hudi** (Uber, nay là Apache)
- → Tìm hiểu cơ chế cốt lõi của từng công nghệ, bắt đầu với Delta Lake

# Delta Lake — table format của Databricks

- Ra đời tại Databricks, sau đó mã nguồn mở hóa hoàn toàn — xây dựng **trên nền Parquet** có sẵn (không thay thế Parquet, mà bổ sung một lớp quản lý bên trên)
- Ý tưởng cốt lõi: mỗi bảng Delta là một thư mục chứa **các file Parquet** + một thư mục con đặc biệt `_delta_log/` ghi lại **lịch sử mọi thay đổi**
- Nguồn: tài liệu chính thức Delta Lake (docs.delta.io) và repo `delta-lake-examples` đã tải về

# Delta Lake — Transaction Log (_delta_log)




_delta_log/ ghi tuần tự các commit — đọc lại toàn bộ log là cách suy ra trạng thái bảng tại bất kỳ thời điểm nào (**time travel**)

*Mỗi thao tác ghi (INSERT/UPDATE/DELETE) tạo một file JSON commit mới trong _delta_log/ — đọc lại toàn bộ log suy ra trạng thái bảng*

# Delta Lake — cơ chế Transaction Log hoạt động ra sao?

- Mỗi thao tác ghi (thêm file, sửa, xóa) tạo ra một file JSON **commit mới**, đánh số tuần tự: `000000.json`, `000001.json`, `000002.json`...
- Mỗi file commit ghi lại **những file Parquet nào được thêm vào, những file nào bị đánh dấu xóa — không sửa trực tiếp** file Parquet đã có (immutable)
- Để biết **trạng thái hiện tại** của bảng, engine chỉ cần đọc **toàn bộ log từ đầu** (hoặc từ checkpoint gần nhất) và "phát lại" (replay) các thay đổi — giống nguyên lý một **sổ cái kế toán** (ledger)

# Delta Lake — ba tính năng chủ chốt

- ACID Transaction
  - Nhiều lượt ghi đồng thời không làm hỏng bảng — mỗi commit là **nguyên tử** (atomic): hoặc ghi trọn vẹn vào log, hoặc không ghi gì cả
- Time Travel
  - Vì log lưu lại **toàn bộ lịch sử**, có thể truy vấn bảng **tại một phiên bản/thời điểm bất kỳ trong quá khứ** — hữu ích để debug hoặc khôi phục sau lỗi
- Schema Enforcement & Evolution
  - Enforcement: từ chối ghi dữ liệu **không khớp schema** đã định nghĩa — ngăn "data swamp"
  - Evolution: cho phép **thêm cột mới** một cách có kiểm soát mà không phá vỡ dữ liệu cũ

# Apache Iceberg — table format khởi nguồn từ Netflix

- Ra đời tại Netflix để giải quyết vấn đề **hiệu năng** khi các bảng Hive truyền thống có **quá nhiều partition/file nhỏ** — nay là dự án Apache độc lập
- Cũng dùng cơ chế **snapshot** (tương tự tinh thần transaction log của Delta) — mỗi thay đổi tạo một snapshot mới, hỗ trợ time travel và ACID tương tự
- Điểm khác biệt nổi bật: **Hidden Partitioning** — Iceberg tự quản lý cách phân vùng vật lý dữ liệu, người dùng **không cần biết** cấu trúc partition khi viết truy vấn (khác Hive truyền thống, nơi viết sai điều kiện partition khiến truy vấn quét toàn bộ bảng)
- **Snapshot isolation**: truy vấn đang chạy luôn thấy một snapshot **nhất quán**, không bị ảnh hưởng bởi các ghi đang diễn ra đồng thời

# Apache Hudi — table format khởi nguồn từ Uber

- Ra đời tại Uber, tập trung giải quyết bài toán **upsert** (update + insert) hiệu quả trên Data Lake — ví dụ liên tục cập nhật trạng thái đơn hàng đang giao
- Hỗ trợ hai chế độ bảng: **Copy-on-Write** (ghi lại toàn bộ file khi update — đọc nhanh, ghi chậm hơn) và **Merge-on-Read** (ghi log riêng, gộp khi đọc — ghi nhanh, đọc cần thêm bước gộp)
- Mạnh nhất trong các use case **cập nhật thường xuyên, gần thời gian thực** (near real-time upsert) — liên hệ trực tiếp tới Phần 4 (streaming) sắp học

# So sánh Delta Lake vs. Apache Iceberg vs. Apache Hudi

| Tiêu chí | Delta Lake | Apache Iceberg | Apache Hudi |
|---|---|---|---|
| Xuất phát từ | Databricks | Netflix | Uber |
| Cơ chế theo dõi thay đổi | Transaction log (JSON) | Snapshot metadata | Timeline + log file |
| Điểm mạnh nổi bật | Tích hợp sâu Spark/-Databricks | Hidden partitioning, đa engine | Upsert/real-time tốt nhất |
| Định dạng dữ liệu nền | Parquet | Parquet/ORC/Avro | Parquet/ORC |

# Khi nào KHÔNG cần Lakehouse/table format?

- Nếu dữ liệu **nhỏ**, ít thay đổi, chỉ phục vụ báo cáo nội bộ đơn giản — một Data Warehouse truyền thống (hoặc thậm chí một database quan hệ thông thường) vẫn là lựa chọn hợp lý, không cần độ phức tạp của Lakehouse
- Nếu khối lượng dữ liệu lớn nhưng **không cần ACID/time travel** (ví dụ chỉ lưu log để tra cứu thỉnh thoảng, không bao giờ update/delete) — một Data Lake thuần với Parquet, không cần thêm lớp Delta/Iceberg/Hudi, vẫn đủ dùng và đơn giản hơn để vận hành
- Nguyên tắc chung: **độ phức tạp của giải pháp nên tương xứng với độ phức tạp của bài toán** — không phải cứ "Big Data" là bắt buộc dùng công nghệ phức tạp nhất

# Cả ba đều giải quyết cùng một bài toán gốc

- Dù khác nhau về chi tiết hiện thực, Delta Lake / Iceberg / Hudi đều nhắm tới **cùng một mục tiêu**: mang ACID transaction, schema quản lý được, và time travel vào Data Lake — biến nó thành Lakehouse
- Trong thực tế, việc chọn công nghệ nào thường phụ thuộc **hệ sinh thái đang dùng** (Databricks → Delta Lake là lựa chọn tự nhiên; đa engine/đa cloud → Iceberg thường linh hoạt hơn; nhiều workload upsert liên tục → cân nhắc Hudi)
- Cả ba đều **mã nguồn mở**, đều có repo ví dụ chính thức đã tải về trong `TaiLieuThamKhao/P2_LuuTruPhanTan/` để thực hành

# Vì sao quay lại chủ đề "nhất quán"?

- Ở phần GFS, ta đã thấy: hệ thống chấp nhận trạng thái "consistent nhưng undefined" để đổi lấy hiệu năng — đây **không phải là một trường hợp riêng lẻ**, mà phản ánh một đánh đổi nền tảng của **mọi** hệ thống phân tán
- Đánh đổi này được hình thức hóa thành một định lý nổi tiếng trong lý thuyết hệ phân tán: **định lý CAP**
- Hiểu CAP giúp ta **đọc hiểu** tài liệu của bất kỳ hệ lưu trữ phân tán nào (HDFS, S3, các hệ NoSQL ở Phần 5) một cách có hệ thống, thay vì ghi nhớ rời rạc từng đặc tính

# Định lý CAP (Brewer's Theorem)

Với một hệ thống lưu trữ dữ liệu **phân tán**, không thể đồng thời đảm bảo **cả ba** tính chất sau ở mức tối đa:

- **Consistency (C)** — mọi node đọc đều thấy *cùng một* dữ liệu mới nhất tại mọi thời điểm
- **Availability (A)** — mọi yêu cầu đều nhận được phản hồi (không bị treo/từ chối), kể cả khi có lỗi
- **Partition tolerance (P)** — hệ thống vẫn hoạt động dù mạng bị *chia cắt* (một số node không liên lạc được với nhau)

Vì mạng máy tính **luôn có thể** xảy ra sự cố phân mảnh (P là bắt buộc trong thực tế), lựa chọn thực sự chỉ còn giữa **C** và **A** khi sự cố xảy ra.

# CAP — áp dụng vào các hệ thống đã học

- GFS/HDFS thiên về CP (Consistency + Partition tolerance)
  - Khi Master/NameNode không liên lạc được với đủ Chunkserver/DataNode, hệ thống **thà từ chối ghi** còn hơn để dữ liệu không nhất quán
- Nhiều hệ NoSQL (Phần 5) thiên về AP (Availability + Partition tolerance)
  - Chấp nhận trả về dữ liệu **có thể hơi cũ** (stale) để đảm bảo hệ thống **luôn phản hồi**, không bao giờ từ chối request
- Không có lựa chọn nào "đúng tuyệt đối" — tùy vào yêu cầu nghiệp vụ (ví dụ: hệ thống ngân hàng cần C hơn; hệ thống đếm lượt xem video có thể chấp nhận A hơn)

# Quorum — kỹ thuật cân bằng giữa Consistency và Availability

- Thay vì chọn tuyệt đối C hoặc A, nhiều hệ thống dùng **quorum** (số phiếu tối thiểu) để điều chỉnh linh hoạt
- Với $N$ bản sao: gọi $W =$ số bản sao phải xác nhận **ghi** thành công, $R =$ số bản sao phải được **đọc** trước khi trả kết quả
- Nếu $W + R > N$: đảm bảo **luôn có ít nhất 1 bản sao chung** giữa tập ghi và tập đọc $\Rightarrow$ đọc luôn thấy dữ liệu mới nhất (strong consistency)
- *Ví dụ*: $N = 3$ (giống replication factor của HDFS/GFS!), chọn $W = 2, R = 2 \Rightarrow W + R = 4 > 3$ — đảm bảo nhất quán mà không cần **cả 3** bản sao phản hồi mỗi lượt ghi/đọc

# Đồng thuận phân tán (Consensus) — ai là "chính" khi mạng chia cắt?

- Bài toán tổng quát hơn lease của GFS: làm sao **nhiều node** thống nhất được **một giá trị/quyết định duy nhất** (ví dụ: ai là Active NameNode) ngay cả khi một số node lỗi hoặc mạng chập chờn?
- Đây là bài toán **Consensus** kinh điển trong hệ phân tán — hai thuật toán nổi tiếng nhất: **Paxos** (Lamport) và **Raft** (thiết kế sau, dễ hiểu/dễ cài đặt hơn)
- MIT 6.5840 (Distributed Systems) — một trong các nguồn tham khảo của Phần 2 — có hẳn một lab thực hành cài đặt Raft từ đầu, và một lab khác cài đặt GFS đơn giản hóa
- Cơ chế **Active/Standby NameNode** của HDFS HA (đã học) thực chất dựa trên nguyên lý consensus tương tự để đảm bảo tại một thời điểm chỉ có **đúng một** NameNode đang "Active"

# Ví dụ phân loại hệ thống thực tế theo CAP

| Hệ thống | Thiên hướng CAP | Lý do |
|---|---|---|
| HDFS / GFS | CP | Từ chối ghi khi không đủ Chunkserver/-DataNode phản hồi |
| Google Spanner | CP (gần A cao nhờ hạ tầng mạnh) | Dùng đồng hồ nguyên tử để giảm chi phí đồng bộ |
| Amazon DynamoDB (mặc định) | AP | Ưu tiên luôn phản hồi, cho phép đọc dữ liệu hơi cũ |
| Cassandra (tunable) | Có thể chỉnh CP hoặc AP | Cấu hình được qua tham số quorum $W, R$ |

# Ví dụ số: quorum ảnh hưởng độ trễ ra sao?

Với $N = 3$ bản sao, giả sử mỗi bản sao phản hồi sau khoảng $10$ms (cùng trung tâm dữ liệu) hoặc $100$ms (khác vùng địa lý).

**Chọn** $W = 1$ (chỉ cần 1 bản sao xác nhận ghi): độ trễ ghi $\approx 10$ms — nhanh, nhưng **không đảm bảo** đọc ngay sau đó luôn thấy dữ liệu mới (rủi ro đọc phải bản sao chưa cập nhật).

**Chọn** $W = 3$ (cần cả 3 bản sao xác nhận, kể cả bản ở vùng xa): độ trễ ghi $\approx 100$ms — chậm hơn 10 lần, nhưng đảm bảo **mọi** bản sao đều nhất quán trước khi xác nhận thành công.

Đây là minh họa định lượng cho đánh đổi CAP: **tăng W/R để tăng Consistency luôn đi kèm cái giá là tăng độ trễ (giảm Availability cảm nhận được)**.

# Tổng kết: một sợi chỉ xuyên suốt từ GFS đến CAP

- Mô hình "Defined/Consistent nhưng undefined" của GFS → là một **biểu hiện cụ thể** của đánh đổi CAP
- Cơ chế **lease** của GFS (chỉ 1 primary được quyết định thứ tự ghi) → là một dạng **đơn giản hóa** của bài toán consensus tổng quát
- Cơ chế **Active/Standby NameNode** của HDFS HA → cần **consensus thực sự** để tránh tình huống "split-brain" (hai node cùng nghĩ mình là Active)
- ⇒ Những khái niệm tưởng như tách rời (replication, lease, HA, CAP) thực chất đều là các mảnh ghép của **cùng một bài toán**: làm sao nhiều máy tính phối hợp đáng tin cậy khi lỗi là điều tất yếu

# Metadata Catalog — "biết mình có gì" trong Lakehouse

- Data Lake/Lakehouse có thể chứa **hàng nghìn bảng**, hàng triệu file — nếu không có nơi **tra cứu tập trung** (bảng nào tồn tại, schema ra sao, ai sở hữu), rất dễ quay lại tình trạng "data swamp" đã học
- **Metadata Catalog** (hay Data Catalog) là thành phần lưu trữ **siêu dữ liệu về siêu dữ liệu**: danh sách bảng, schema, vị trí file, thống kê, quyền truy cập
- Hai catalog phổ biến trong hệ sinh thái Hadoop/Spark: **Hive Metastore** (truyền thống, on-premise) và **AWS Glue Data Catalog** (managed, tích hợp sẵn trên AWS)

# Data Lineage — theo dõi "dữ liệu này từ đâu ra"

- **Data lineage** (phả hệ dữ liệu) ghi lại **đường đi** của dữ liệu: từ nguồn thô, qua các bước biến đổi (ETL), đến bảng/báo cáo cuối cùng
- Quan trọng khi cần **debug** (một số liệu sai — lỗi xảy ra ở bước biến đổi nào?), hoặc **tuân thủ quy định** (compliance — chứng minh dữ liệu cá nhân được xử lý đúng quy trình)
- Các bảng Lakehouse (Delta/Iceberg) với transaction log/snapshot đã học tự nhiên hỗ trợ một phần lineage — biết chính xác **phiên bản nào** của bảng được dùng để tạo ra kết quả nào

# Access Control — ai được phép làm gì với dữ liệu?

- Khác với một file hệ thống truyền thống (quyền đọc/ghi theo user Linux), Lakehouse cần kiểm soát quyền **chi tiết hơn**: theo bảng, theo cột (ví dụ ẩn cột lương), thậm chí theo dòng (row-level security)
- Nguyên tắc **least privilege** (đặc quyền tối thiểu): mỗi người/hệ thống chỉ được cấp đúng quyền cần thiết cho công việc, không hơn
- Đây là lớp **governance** (quản trị) — một trong ba tính năng DBMS truyền thống mà định nghĩa Lakehouse (Armbrust et al.) nhấn mạnh phải có, bên cạnh ACID và indexing

# Ba nguồn dữ liệu governance thường gặp trong Lakehouse

| Thành phần | Vai trò | Ví dụ công cụ |
|---|---|---|
| Metadata Catalog | Tra cứu bảng/schema tập trung | Hive Metastore, AWS Glue Catalog |
| Data Lineage | Theo dõi nguồn gốc & biến đổi dữ liệu | OpenLineage, Marquez |
| Access Control | Kiểm soát quyền đọc/ghi chi tiết | IAM policy, Ranger, Unity Catalog |

# Governance trong Lakehouse — vì sao khó hơn Data Warehouse truyền thống?

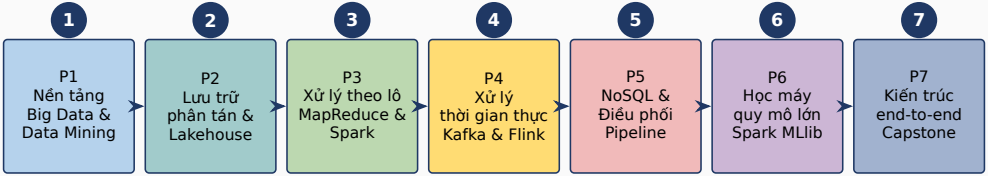
- Data Warehouse: dữ liệu tập trung trong **một hệ quản trị duy nhất** — áp quyền hạn tương đối đơn giản qua SQL GRANT/REVOKE
- Lakehouse: dữ liệu là **file thô trên object storage**, có thể được **nhiều engine khác nhau** truy cập trực tiếp (Spark, Presto/Trino, Python thuần) — nếu chỉ áp quyền ở tầng SQL engine, ai đó vẫn có thể đọc thẳng file Parquet mà bỏ qua kiểm soát
- Giải pháp thực tế: kết hợp quyền hạn ở **tầng lưu trữ** (IAM policy trên bucket/thư mục) **và tầng catalog/table format** (Iceberg/Delta hỗ trợ một số cơ chế kiểm soát truy cập tích hợp) — đây vẫn là một lĩnh vực đang phát triển nhanh

# Governance & quy định pháp lý về dữ liệu cá nhân

- Nhiều quốc gia có luật riêng về bảo vệ dữ liệu cá nhân (ví dụ GDPR ở Châu Âu; tại Việt Nam là **Nghị định về Bảo vệ dữ liệu cá nhân**) — yêu cầu tổ chức phải biết rõ **dữ liệu cá nhân nào đang lưu, ở đâu, ai truy cập được**
- Yêu cầu thường gặp: quyền **xóa dữ liệu theo yêu cầu** (right to be forgotten) — đây là lý do các table format có ACID (Delta/Iceberg/Hudi) có lợi thế lớn so với Data Lake thô: có thể **xóa chính xác** bản ghi của một người dùng mà không phải viết lại toàn bộ bảng
- $\Rightarrow$ Governance không chỉ là vấn đề kỹ thuật "cho gọn gàng" — mà ngày càng là **yêu cầu tuân thủ pháp lý bắt buộc** khi thiết kế hệ thống lưu trữ dữ liệu người dùng

# CASE STUDY: THIẾT KẾ LƯU TRỮ CHO MỘT NỀN TẢNG GỌI XE & GIAO ĐỒ ĂN

*Áp dụng toàn bộ lý thuyết Phần 2 vào một bài toán thực tế, xuyên suốt (giống case study Tuần 2)*



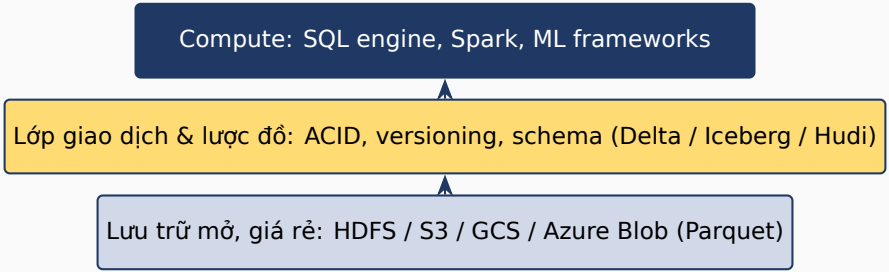
# Bối cảnh: đặc điểm dữ liệu của nền tảng

- Ba luồng dữ liệu chính: (1) **vị trí GPS tài xế** — hàng triệu bản ghi nhỏ mỗi phút (Velocity cao, Volume lớn), (2) **đơn hàng/chuyến đi** — bán cấu trúc, tần suất vừa phải, (3) **log ứng dụng & sự kiện người dùng** — khối lượng lớn, đa dạng định dạng
- Yêu cầu nghiệp vụ: vừa cần **báo cáo/BI** (doanh thu theo ngày, theo khu vực) vừa cần **huấn luyện ML** (dự đoán thời gian đến, gợi ý tài xế) trên **cùng một nguồn dữ liệu**
- $\Rightarrow$ Đây chính xác là bài toán mà kiến trúc **Lakehouse** được thiết kế để giải quyết

# Quyết định 1: Lưu trữ vật lý — HDFS on-premise hay Object Storage?

- Với một nền tảng khởi nghiệp/scale nhanh, khối lượng dữ liệu tăng **không dự đoán trước được** — ưu tiên **tách compute/storage** để mở rộng linh hoạt mà không đầu tư phần cứng trước
- $\Rightarrow$ Chọn **Object Storage** (S3/GCS) làm lớp lưu trữ nền tảng thay vì tự vận hành cụm HDFS
- Chỉ cân nhắc HDFS on-premise nếu có ràng buộc **data residency** nghiêm ngặt (ví dụ quy định dữ liệu tài xế/khách hàng phải lưu trong lãnh thổ quốc gia, không được ra nước ngoài)

# Quyết định 2: Kiến trúc phân lớp Bronze – Silver – Gold (Medallion Architecture)




Lakehouse = Data Lake (lưu trữ mở, rẻ) + lớp quản lý giao dịch của Data Warehouse

*Mẫu kiến trúc Lakehouse phổ biến nhất trong thực tế: dữ liệu đi qua 3 lớp chất lượng tăng dần, mỗi lớp là một bảng Delta/Iceberg riêng*

# Bronze Layer — lớp dữ liệu thô

- Nạp **nguyên bản** dữ liệu từ nguồn (GPS ping, log sự kiện, đơn hàng) — **không biến đổi**, giữ đúng định dạng gốc từ hệ thống nguồn
- Định dạng: thường dùng **Avro** hoặc JSON — vì dữ liệu đang được ghi **liên tục, theo dòng sự kiện** (streaming), ưu tiên tốc độ ghi hơn tốc độ đọc phân tích
- Mục đích: là "nguồn sự thật" (source of truth) để có thể **tính toán lại từ đầu** nếu logic xử lý ở các lớp sau có lỗi

# Silver Layer — lớp dữ liệu đã làm sạch

- Dữ liệu từ Bronze được **lọc bản ghi lỗi, chuẩn hóa kiểu dữ liệu, khử trùng lặp** (deduplication) — ví dụ loại bỏ các GPS ping trùng lặp do lỗi mạng gửi lại
- Chuyển đổi định dạng sang **Parquet**, tổ chức theo bảng Delta/Iceberg với **partition theo ngày** — bắt đầu tối ưu cho truy vấn phân tích
- Đây là lớp mà hầu hết **data engineer** làm việc hàng ngày: join các nguồn dữ liệu khác nhau, xây dựng bảng trung gian

# Gold Layer — lớp dữ liệu sẵn sàng phục vụ

- Dữ liệu đã được **tổng hợp** (aggregate) theo nhu cầu nghiệp vụ cụ thể: doanh thu theo ngày/khu vực, đặc trưng (feature) sẵn sàng cho mô hình ML
- Tối ưu mạnh cho **tốc độ đọc**: partition hợp lý, có thể thêm chỉ mục phụ trợ, kích thước file lớn (tránh vấn đề nhiều file nhỏ)
- Đây là lớp mà **BI dashboard** và **pipeline huấn luyện ML** (liên hệ Phần 6) trực tiếp đọc vào — không bao giờ đọc thẳng từ Bronze

# Quyết định 3: Chọn table format nào cho từng lớp?

- Lớp **Silver** (bảng vị trí GPS tài xế): cần **upsert liên tục** — vị trí tài xế thay đổi mỗi vài giây, cần cập nhật "trạng thái mới nhất" hiệu quả ⇒ ưu tiên **Apache Hudi** (thế mạnh upsert/near real-time)
- Lớp **Gold** (báo cáo doanh thu, feature cho ML): dữ liệu ít thay đổi sau khi tổng hợp, cần **time travel** để audit số liệu báo cáo ⇒ **Delta Lake** hoặc **Iceberg** đều phù hợp
- Bài học quan trọng: **không nhất thiết chỉ dùng một table format duy nhất** cho toàn bộ hệ thống — lựa chọn nên dựa trên **đặc điểm workload của từng lớp dữ liệu**

# Quyết định 4: Chiến lược Partition

- Bảng GPS tài xế (Silver): partition theo **ngày + khu vực (thành phố)** — cân bằng giữa số lượng partition hợp lý và khả năng partition pruning hiệu quả cho truy vấn theo khu vực
- **Không** partition theo `driver_id` — sẽ tạo ra hàng trăm nghìn partition nhỏ, rơi vào vấn đề "nhiều file nhỏ" đã học
- Bảng doanh thu (Gold): partition theo **tháng** — vì truy vấn báo cáo thường theo tháng/quý, partition quá mịn (theo ngày) sẽ không mang lại lợi ích tương xứng với chi phí quản lý thêm

# Quyết định 5: Chịu lỗi & sao lưu

- Object storage (S3/GCS) đã tự đảm bảo **độ bền rất cao** (thường 99.999999999%) trong **một vùng (region)** — nhưng vẫn cần chiến lược cho rủi ro **mất cả một region** (thiên tai, sự cố lớn)
- Với dữ liệu quan trọng (đơn hàng, giao dịch thanh toán): bật **cross-region replication** — tự động sao chép sang region khác, đánh đổi thêm chi phí lưu trữ để giảm rủi ro tối đa
- Với dữ liệu Bronze (thô, có thể tính lại từ nguồn): có thể chấp nhận **mức bảo vệ thấp hơn** để tiết kiệm chi phí — không phải mọi dữ liệu đều cần cùng một mức chịu lỗi

# Quyết định 6: Governance cho dữ liệu nhạy cảm

- Vị trí GPS và thông tin tài xế/khách hàng là **dữ liệu cá nhân nhạy cảm** — cần **access control chi tiết theo cột** (ví dụ: chỉ đội ngũ vận hành mới xem được vị trí thời gian thực, đội phân tích chỉ thấy dữ liệu đã ẩn danh)
- Metadata catalog (Hive Metastore/Glue) ghi rõ **chủ sở hữu** và **mức độ nhạy cảm** của từng bảng — là điều kiện tiên quyết để áp policy truy cập tự động
- Data lineage giúp trả lời nhanh câu hỏi kiểm toán: "báo cáo doanh thu tháng này lấy dữ liệu từ những nguồn nào, qua bước xử lý nào?"

# Tổng hợp kiến trúc lưu trữ của case study

| Lớp | Định dạng | Table format | Partition |
|---|---|---|---|
| Bronze | Avro/JSON | Không (raw) | Theo giờ nạp |
| Silver (GPS) | Parquet | Apache Hudi (upsert) | Ngày + khu vực |
| Silver (đơn hàng) | Parquet | Delta Lake | Ngày |
| Gold (báo cáo/feature) | Parquet | Delta Lake / Iceberg | Tháng |

# Bài học rút ra từ case study

- Không có **một câu trả lời đúng duy nhất** — kiến trúc lưu trữ luôn là kết quả của việc **cân bằng nhiều đánh đổi**: chi phí, độ trễ, độ tin cậy, tính linh hoạt
- Mọi quyết định trong case study này đều **bắt nguồn trực tiếp** từ các khái niệm lý thuyết đã học: replication (Quyết định 5), partitioning/small-files (Quyết định 4), table format trade-off (Quyết định 3), CAP/consistency (ẩn trong lựa chọn Hudi cho upsert)
- Đây chính là kỹ năng quan trọng nhất của một Data/Big Data Engineer: **áp dụng đúng nguyên lý vào đúng bối cảnh**, không phải học thuộc lòng công cụ

# Tổng kết lý thuyết Phần 2 — bức tranh toàn cảnh

- Lưu trữ phân tán ở tầng thấp
  - HDFS & GFS — kiến trúc master/worker, block/chunk, replication, chịu lỗi
- Định dạng & tổ chức file
  - Row-based vs. Columnar (Parquet/ORC), partitioning
- Lưu trữ trên Cloud
  - Object Storage (S3/GCS/Azure) — tách compute khỏi storage
- Tầng quản lý dữ liệu cao hơn
  - Data Warehouse $\rightarrow$ Data Lake $\rightarrow$ Lakehouse; Delta Lake, Iceberg, Hudi
- Nguyên lý nền tảng xuyên suốt
  - CAP theorem, quorum, consensus, governance

# Những điểm mấu chốt cần nhớ

**Những điểm mấu chốt:**

- HDFS/GFS giải quyết đồng thời scale-out **và** chịu lỗi — không thể tách rời hai mục tiêu này
- Columnar storage (Parquet/ORC) thay đổi **căn bản** chi phí I/O nhờ column pruning và predicate pushdown
- Lakehouse giải quyết mâu thuẫn Data Warehouse (đắt, cứng nhắc) vs Data Lake (rẻ, thiếu quản trị) bằng transaction log
- CAP là **sợi chỉ đỏ** giải thích các đánh đổi từ GFS đến NoSQL (Phần 5)

## Câu hỏi ôn tập

1. Vì sao HDFS chọn block size 128MB thay vì 4KB như đĩa cứng truyền thống?
2. Truy vấn chỉ cần 3/50 cột — columnar storage tiết kiệm I/O khoảng bao nhiêu %?
3. Data Lake khác Data Warehouse ở điểm nào về schema? Lakehouse giải quyết ra sao?
4. Với quorum $N = 3, W = 2, R = 2$: vì sao $W + R > N$ đảm bảo đọc thấy dữ liệu mới nhất?

# Tài liệu tham khảo cho Phần 2

| Tài liệu | Nội dung sử dụng trong bài giảng |
|---|---|
| Apache Hadoop — HDFS Architecture Guide (docs chính thức) | Giả định thiết kế, kiến trúc NameNode/-DataNode, block/replication |
| Ghemawat, Gobioff, Leung — *The Google File System* (SOSP 2003) | Kiến trúc Master/Chunkserver, chunk size, lease, mô hình nhất quán |
| Armbrust et al. — *Lakehouse: A New Generation of Open Source Systems and Applications* (CIDR 2021) | Định nghĩa Lakehouse, 3 mục tiêu thiết kế |
| Apache Parquet, Delta Lake, Apache Iceberg — tài liệu chính thức từng dự án | Columnar storage, transaction log, hidden partitioning |
| MIT 6.5840 — Distributed Systems (lab GFS, Raft) | Consensus, tham khảo mở rộng cho phần CAP/quorum |

# Vì sao những nguồn tham khảo này?

- **HDFS Architecture Guide** và tài liệu **Parquet/Delta/Iceberg** là tài liệu **chính thức** từ chính các dự án Apache/Delta.io — đảm bảo thông tin luôn khớp với phiên bản đang được dùng rộng rãi trong công nghiệp
- **Bài báo GFS** và **bài báo Lakehouse** được nhóm biên soạn **tải trực tiếp** từ nguồn công khai chính thức (Google Research, CIDR open-access) — đảm bảo trích dẫn đúng nguyên văn, không suy diễn qua nguồn thứ cấp
- Toàn bộ các nguồn tải được bằng CLI đã được thêm vào script `download_bigdata_resources.sh` — chạy lại script để lấy 2 bài báo mới (GFS, Lakehouse) nếu chưa có

# THỰC HÀNH PHẦN 2

*Áp dụng HDFS/GFS, định dạng file, và Lakehouse vào bài tập tính toán & lập trình cụ thể*

